

Memory and Stack

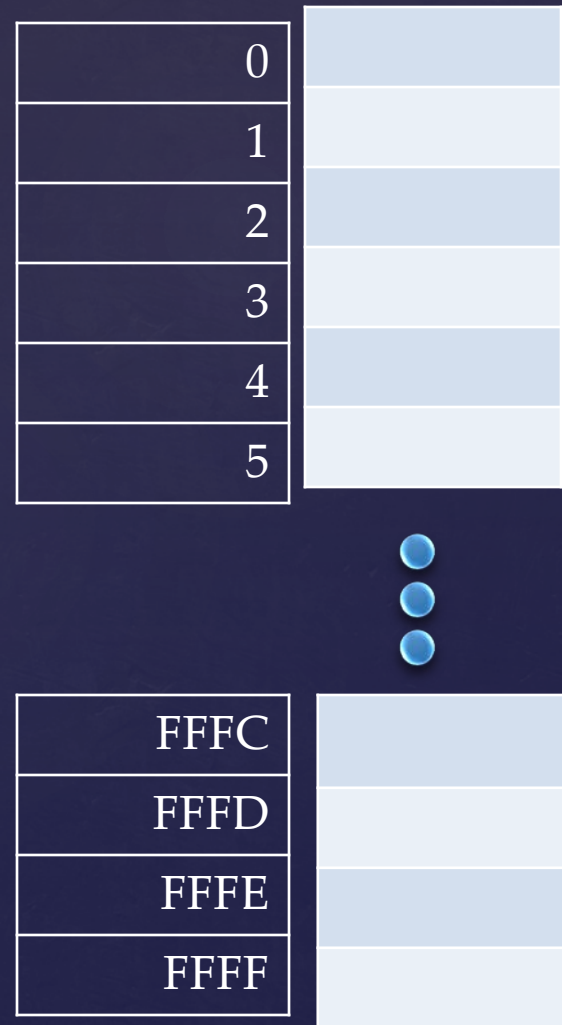
So far we have just assumed “memory” was there, doing its job.

Today we will talk more about what that job is.

Memory Model

To begin, we must remember the basic model of memory.

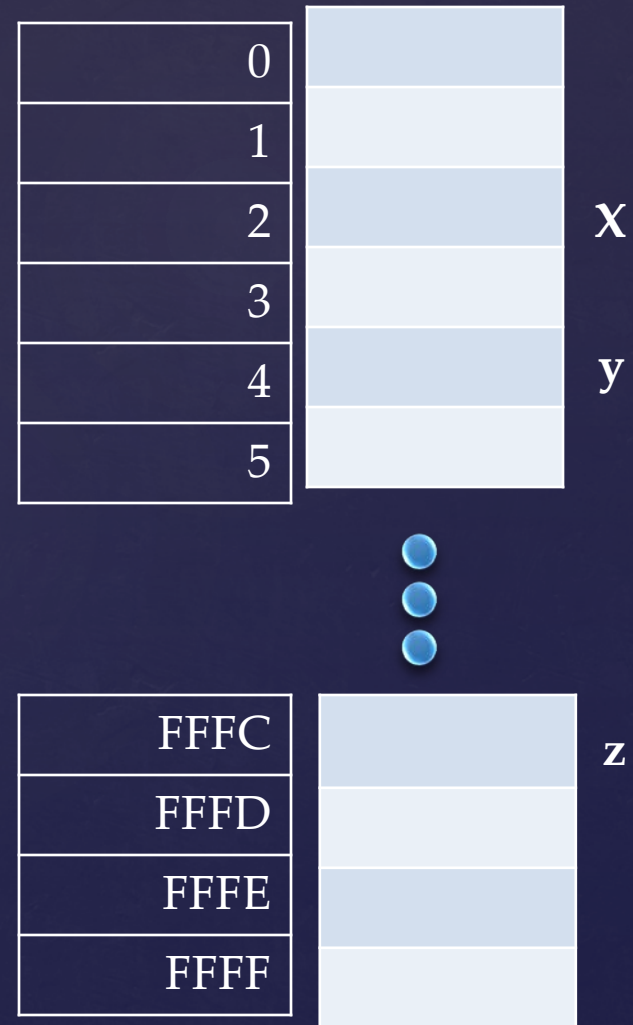
Memory is just a long array of **words**. The **index** of each word is its **address**



Memory Model

The thing we call a **pointer** is just the **address**.

The thing we call a **variable** is just a name for the **address**.

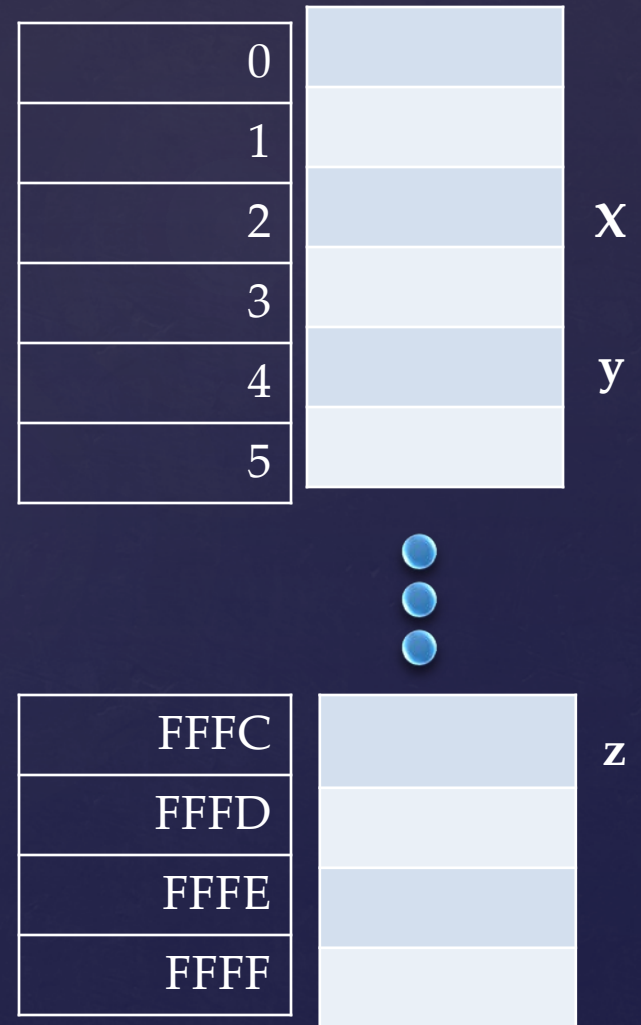


Memory Model

$\&x == 2$

$\&y == y$

$\&z == 0xFFFC$



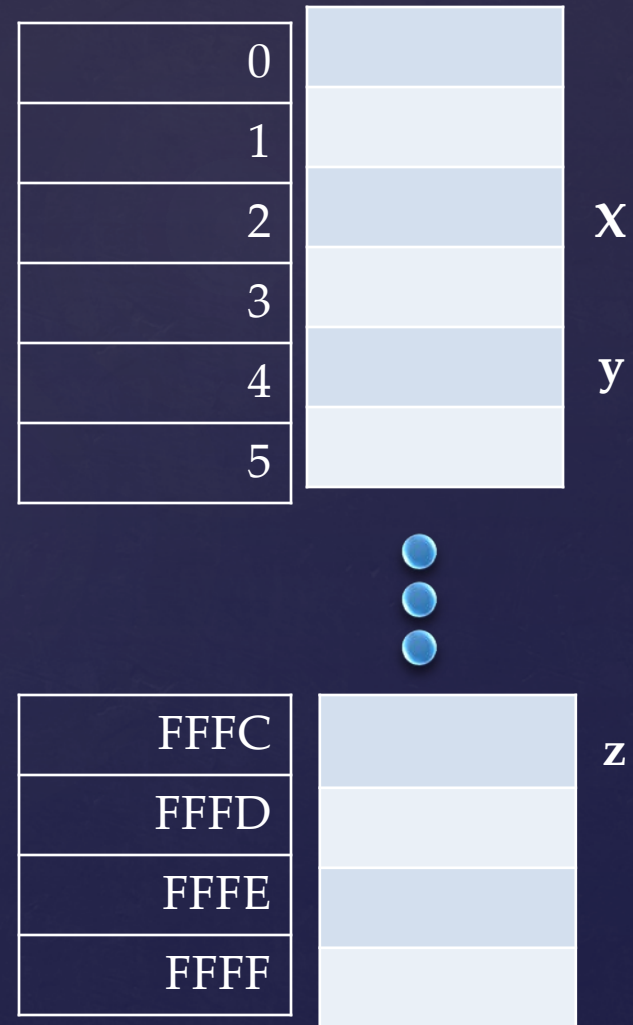
Memory Model

If we write:

$z = x + y;$

The compiler generates:

$*(&z) = *(&x) + *(&y);$



Memory Model

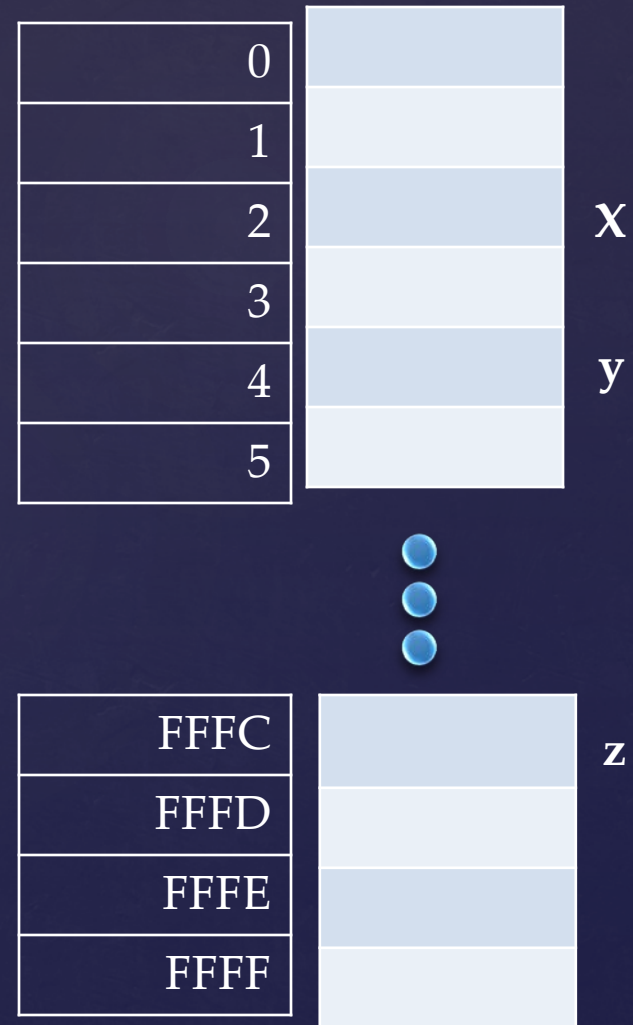
Or, in assembly:

LD R1, \$0002

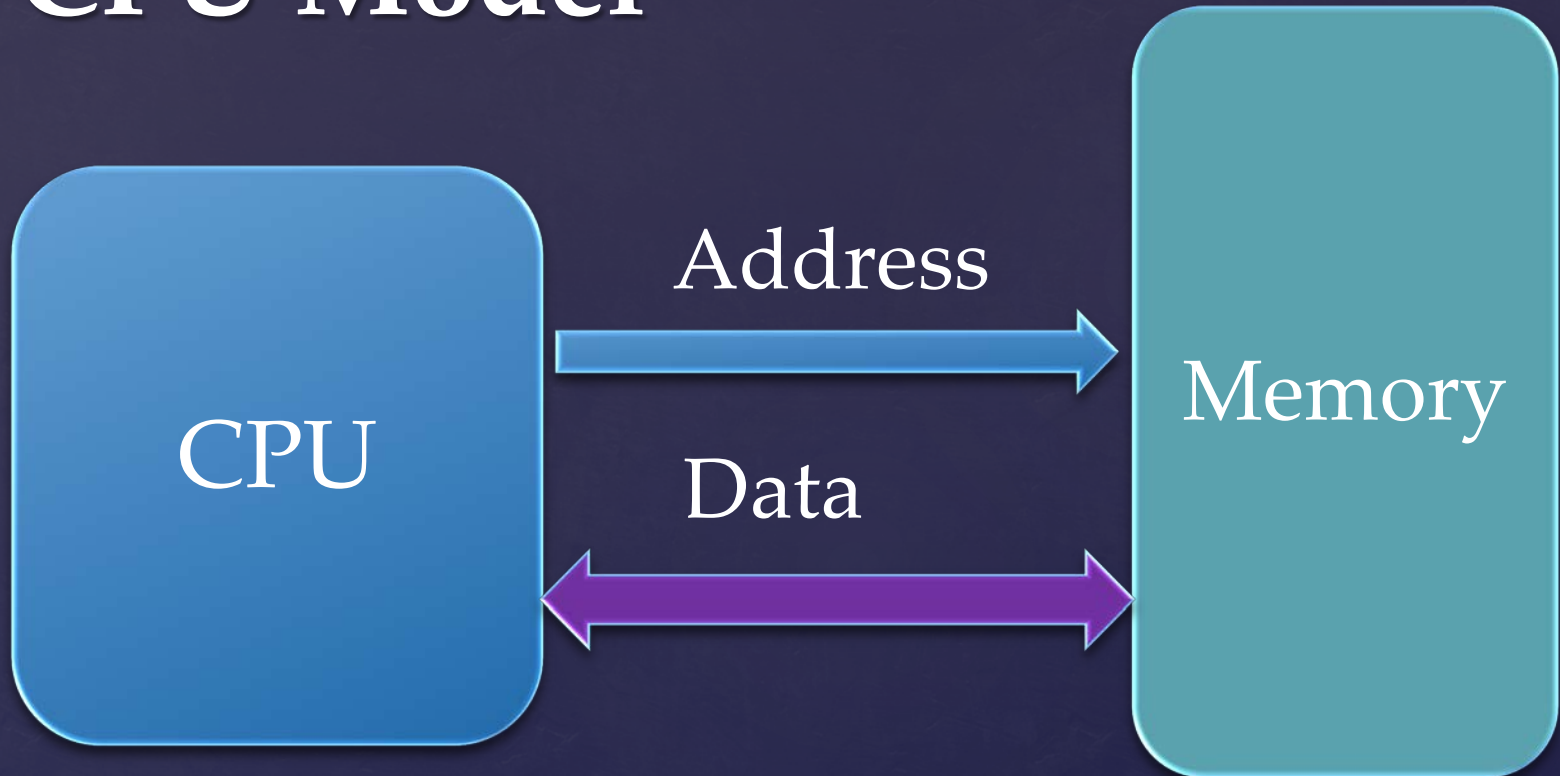
LD R2, \$0004

ADD R3, R1, R2

ST R3, \$FFFC



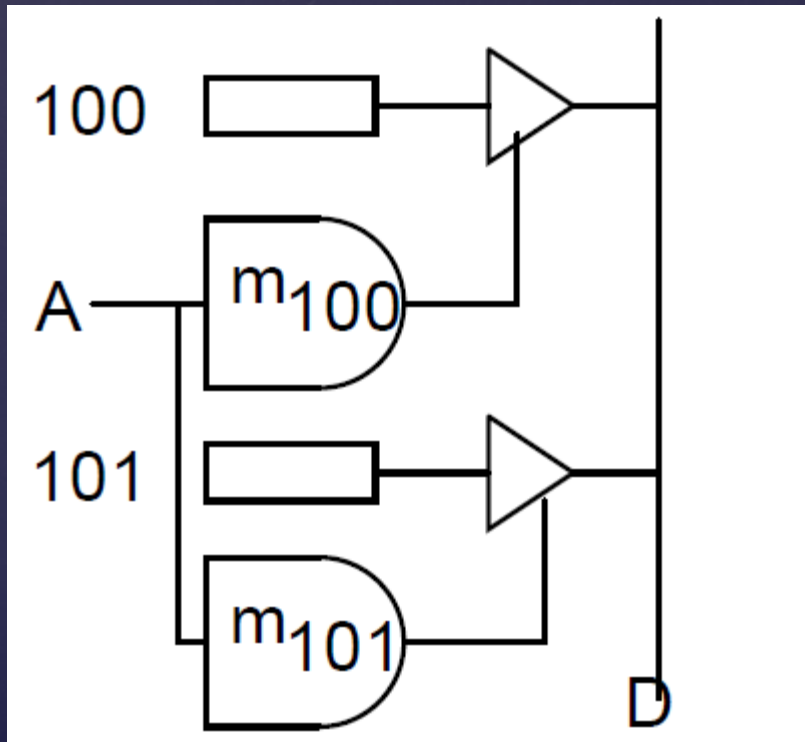
CPU Model



Finally, remember that the CPU and memory are physical machines, connected by wires.

What actually happens

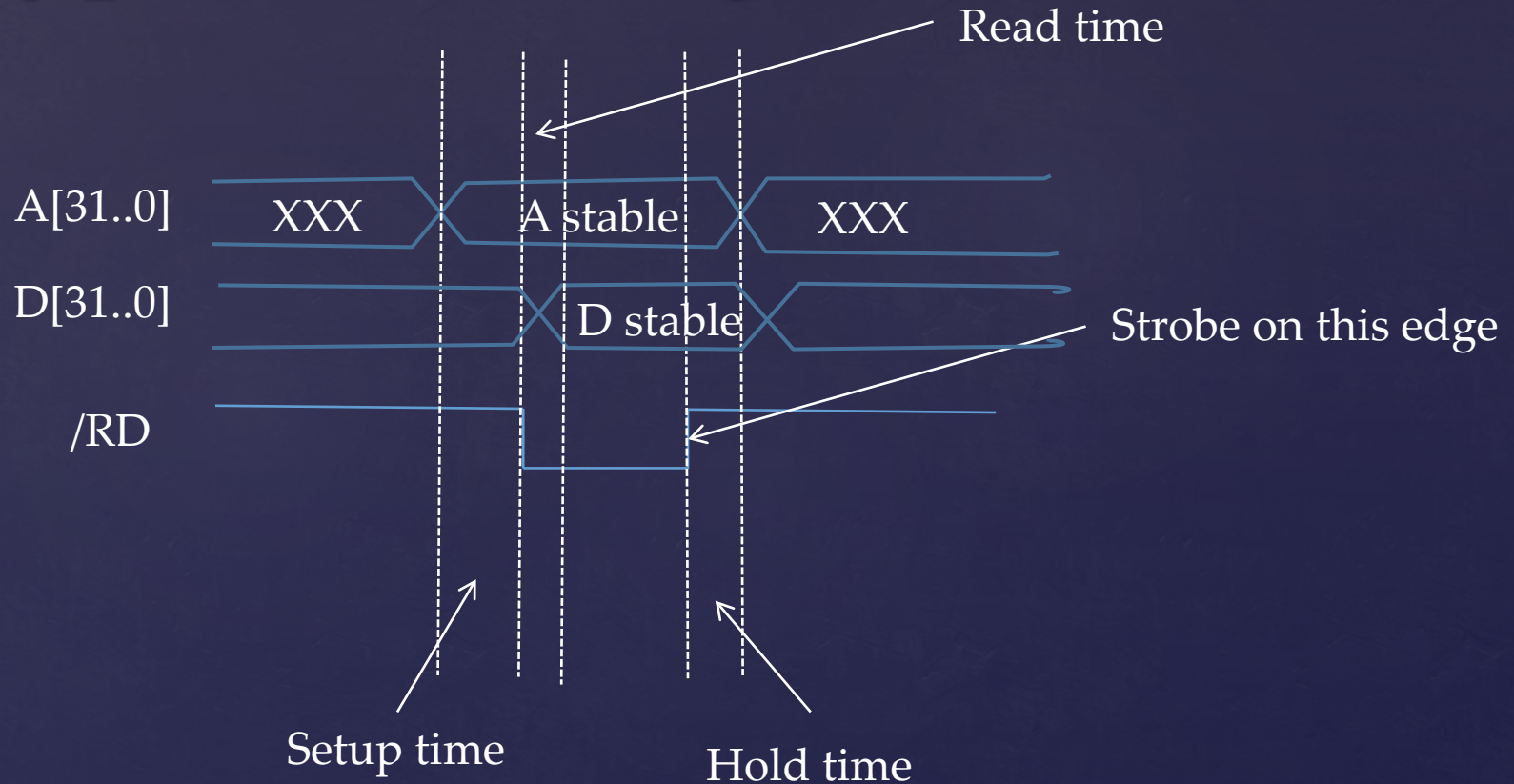
Essentially, a decoder identifies the specific minterm associated with the Address (A) lines and puts the value on the Data (D) lines



This picture shows READ only.

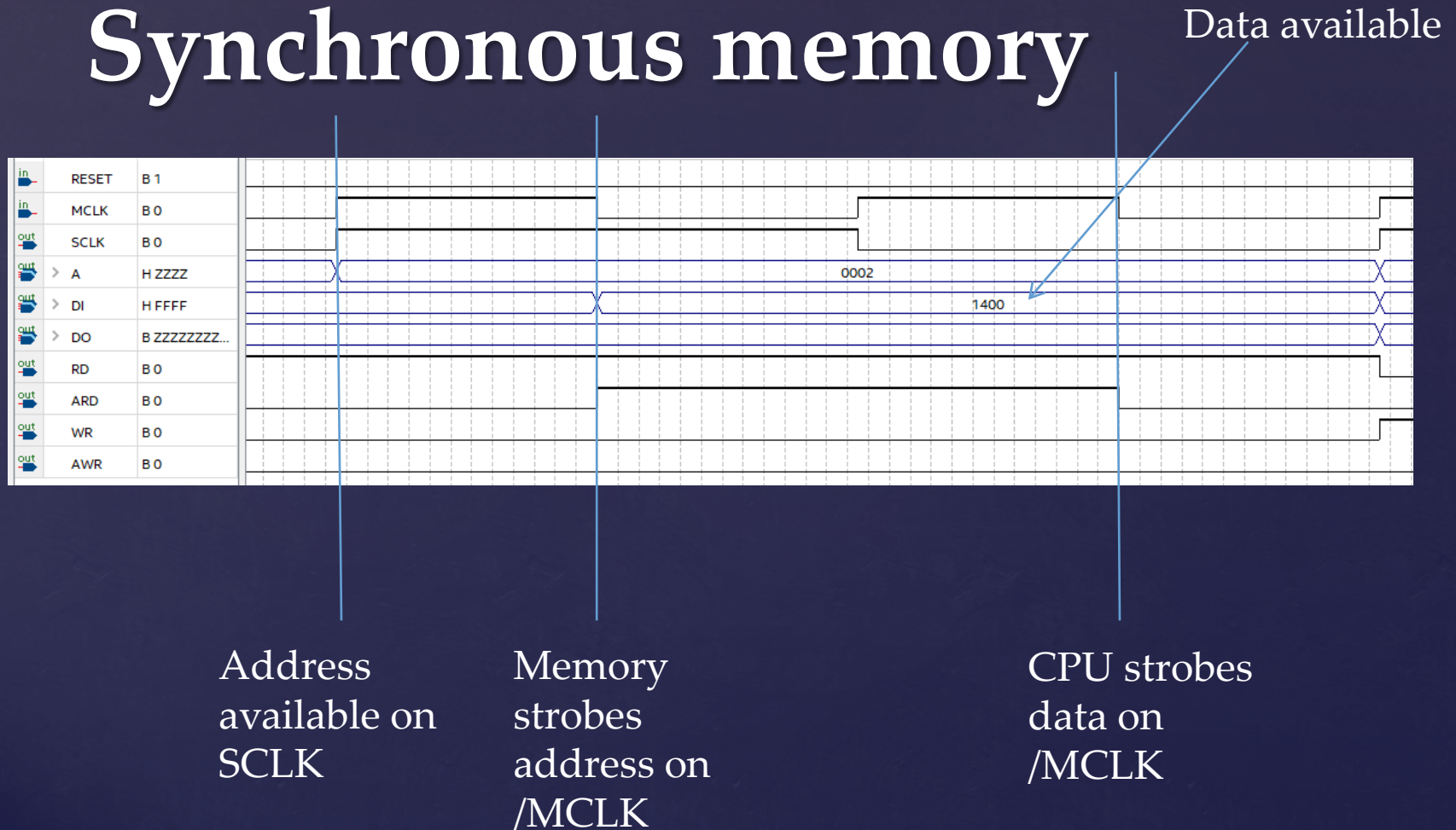
This picture also shows a memory with TWO words of storage. A typical memory has thousands, millions, or billions of cells

Typical Read Cycle

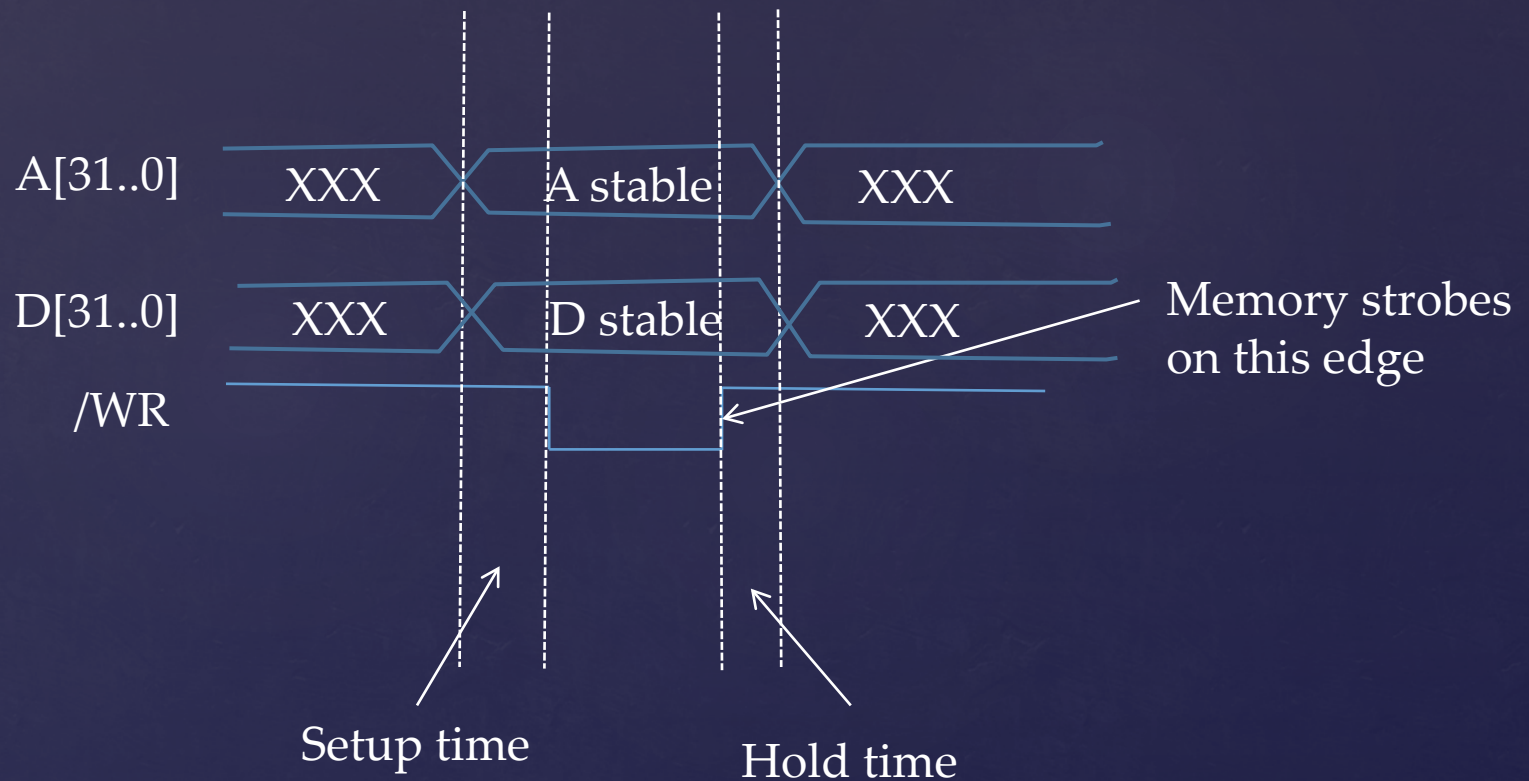


Like everything else, no two systems are alike

Read Cycle with Synchronous memory

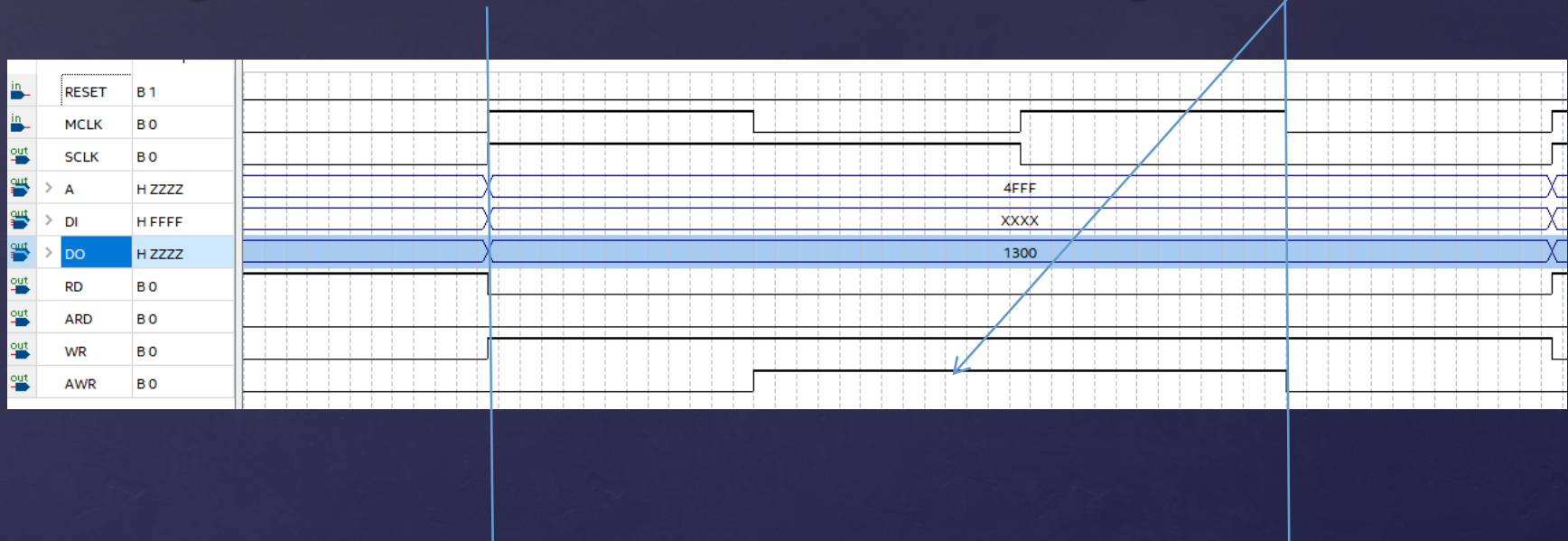


Typical Write Cycle



Like everything else, no two systems are alike

Write Cycle with Synchronous memory



Address &
data
available on
SCLK

Memory
strokes data
on /MCLK

Intro to The Stack

Most CPUs (though not all, as we will see) include a register called the **stack pointer (SP)** and a special area of memory called **“The Stack”**.

We call it “The Stack” because it is a special case of the Last-In-First-Out abstract data storage type called “a stack”. The Stack is a stack, but not all stacks are The Stack.

Stack Operations

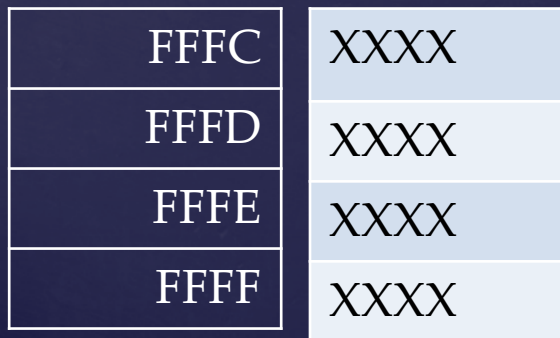
CPUs that support The Stack also support some dedicated stack operations. Usually these are called PUSH and POP.

PUSH is for putting data (usually register contents) on The Stack.

POP is for taking data off The Stack (and putting it into a register).

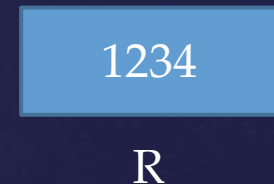
For reasons lost in time, The Stack “grows downward”. This means that as you add elements to it, the value in the stack pointer (SP) decreases.

Example of PUSH



Let's assume our SP points at the next empty spot in The Stack.

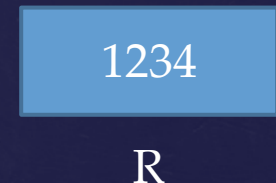
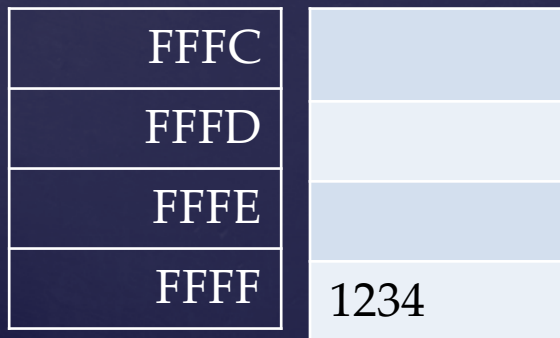
Here is the situation **before** PUSH R



Example of PUSH



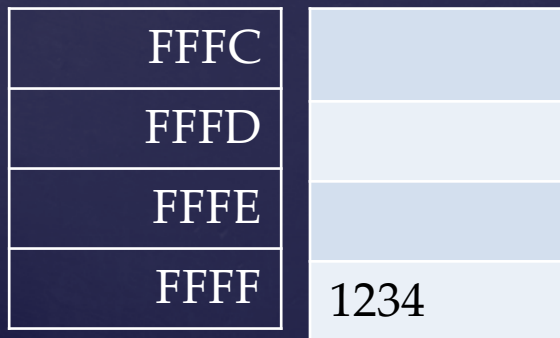
Here is the situation **after** PUSH R



Example of POP



Here is the situation **before** POP R



Example of POP

0	
1	
2	
3	
4	
5	



FFFC	XXXX
FFFD	XXXX
FFFE	XXXX
FFFF	1234

Here is the situation **after** POP R

Note the data is “still there” in memory, but NOT on The Stack

FFFF

SP

1234

R

The Stack and subroutines

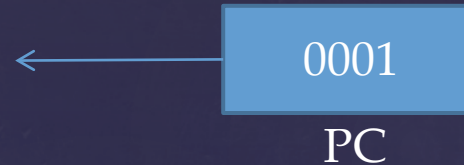
As we have seen, at the core of CPU operation is the operation called **Fetch** (get the next instruction). Normally the “next” instruction is the next one in the line. That is, if we are currently executing the instruction at PC, the next instruction is at PC+1.

We have already seen execution for **branch** instructions, in which the execution of the instruction **forces** the PC to **point** (and thus **jump**) somewhere else.

0	
1	ADD R1,R2,R3
2	BRA 5
3	Inst
4	Inst
5	ADD R4,R5,R6



FFFC	XXXX
FFFD	XXXX
FFFE	XXXX
FFFF	1234



A normal instruction fetch just moves us down

0	
1	ADD R1,R2,R3
2	BRA 5
3	Inst
4	Inst
5	ADD R4,R5,R6



FFFC	XXXX
FFFD	XXXX
FFFE	XXXX
FFFF	1234

0002
PC

But a branch causes us to jump out of the sequence

Subroutines

This is all fine, but with subroutines, we don't just want to jump **there**. We want to jump **back**.

C-code example:

```
...  
Func();  
...
```

```
void Func()  
{  
...  
}
```

0
1
2
3
4
5

FFFC
FFFD
FFFE
FFFF

	...
	JSR FUNC
	...



FUNC	...
	...
	RTS



JSR and RTS

The traditional solution to this problem is to use The Stack to store the **return address**. The return address (**RA**) is **what would have been** the next instruction, if not for the branch, ie: **PC**. So we traditionally have these two instructions:

Opcode	Microoperations
JSR #F	MAR $\leftarrow$ PC
	INC PC
	PUSH PC
	PC $\leftarrow$ M[MAR]

Opcode	Microoperations
RTS	POP PC

As we will see, some modern CPUs manage to do this **without The Stack**, but they must behave **as if** they have The Stack

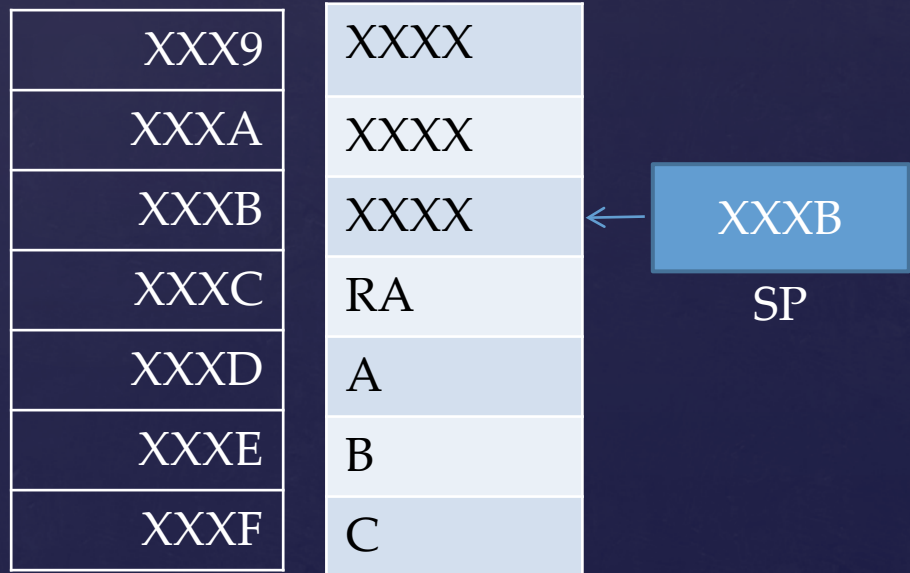
The stack frame

Now we have two new problems. Problem #1 is that users want to pass parameters to our subroutines. That is, users want to write things like this:

Func(a,b,c); // Where should we put a,b,c?

The traditional place is The Stack. Suppose that before the call, SP=XXXF. Then:

C-code	Assembly
Func(a,b,c);	PUSH C
	PUSH B
	PUSH A
	JSR FUNC



The stack frame: parameters

The nice thing about this trick is that from the subroutine's perspective, the parameters are **always** "in the **same** place". The stack pointer moves around, but SP+1, SP+2, SP+3... are **unambiguous**.

C-code	Assembly
Func(a,b,c);	PUSH C
	PUSH B
	PUSH A
	JSR FUNC

XXX9	XXXX	
XXXA	XXXX	
XXXB	XXXX	← SP
XXXC	RA	← SP+1
XXXD	A	← SP+2
XXXE	B	← SP+3
XXXF	C	← SP+4

The stack frame: locals

In high level languages, subroutines can also have **local variables** and **temporary variables**.

Local variables are variables the user declares and uses that exist only while the function is executing.

Temporary variables are variables the user doesn't know about, but the compiler needs.

```
void Func(int a, int b, int c)
{
    int x;
    int y;
    int z;

    ...
    x = a+b+c;
}
```

Locals



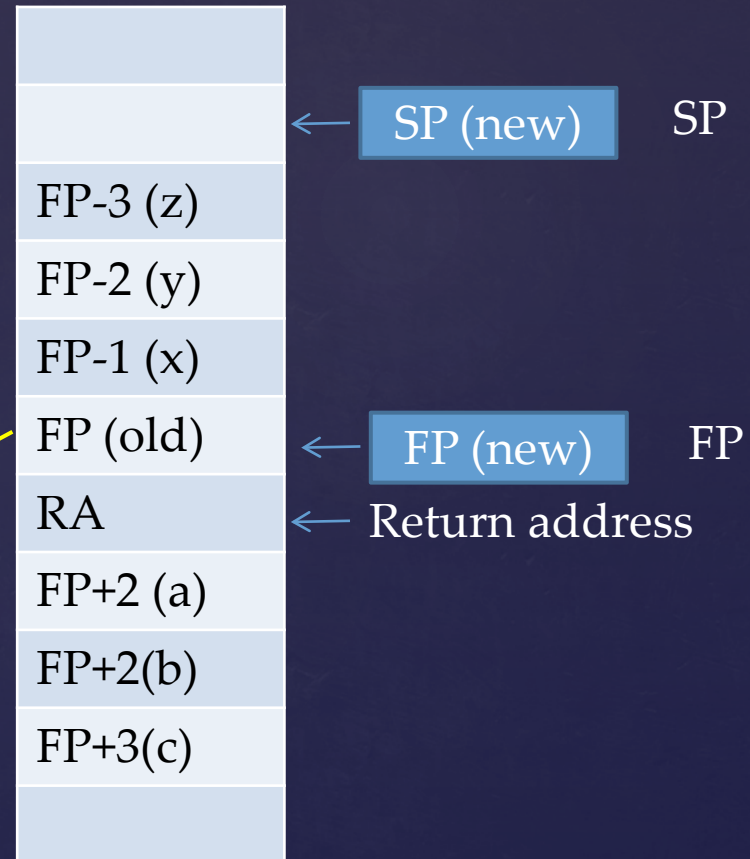
b+c is probably temporary



The stack frame: locals

So where do they go? The Stack, of course. The traditional approach is to set up a **stack frame** using the **frame pointer** (Intel calls this the **base pointer**). A typical stack frame looks like this:

Somewhere down there is the stack frame of the caller.



LINK and UNLINK

To simplify this process, many CPUs support **LINK** and **UNLINK**. They set up and destroy the stack frame. For those of you who are interested, the **linked list** made by FP is what is **unrolled** by **exceptions**.

Opcode	Microoperations
LINK #n	$\text{MDR} \leftarrow \text{M}[\text{PC}++]$
	PUSH FP
	$\text{FP} \leftarrow \text{SP}$
	$\text{SP} \leftarrow \text{SP} - \text{MDR}$

Opcode	Microoperations
UNLINK	$\text{SP} \leftarrow \text{FP}$
	POP FP

As we will see, some modern CPUs manage to do this **without The Stack**, but they must behave **as if** they have The Stack